



Security Assessment Report

Exponent Tranching

June 22, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the Exponent Tranching.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 8 issues or questions.

Task	Type	Commit
exponent_tranching	Solana	b45838d (05/25/2026)

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview	3
Findings in Detail	4
[L-01] Senior loss settlement can overdraw junior token bucket	4
[L-02] First accrued junior return share can be incorrectly reset	9
[L-03] Accumulation of rounding dust can cause DoS	11
[I-01] Small dust deposits and withdrawals can distort LP price	14
[I-02] Circumventing fees by splitting withdrawals	17
[I-03] Inconsistencies in the ModifyMarket instruction	21
[I-04] Suboptimal numerical integration for target drift calculation	23
[I-05] Redundant Token-2022 extension check	24
Appendix: Methodology and Scope of Work	26

Result Overview

Issue	Impact	Status
EXPONENT_TRANCHING		
[L-01] Senior loss settlement can overdraw junior token bucket	Low	Resolved
[L-02] First accrued junior return share can be incorrectly reset	Low	Resolved
[L-03] Accumulation of rounding dust can cause DoS	Low	Resolved
[I-01] Small dust deposits and withdrawals can distort LP price	Info	Resolved
[I-02] Circumventing fees by splitting withdrawals	Info	Resolved
[I-03] Inconsistencies in the ModifyMarket instruction	Info	Resolved
[I-04] Suboptimal numerical integration for target drift calculation	Info	Resolved
[I-05] Redundant Token-2022 extension check	Info	Resolved

Findings in Detail

EXPONENT_TRANCHING

[L-01] Senior loss settlement can overdraw junior token bucket

Identified in commit [b45838d](#).

`update_tranching_market` can create a mismatch between junior effective NAV and the SY actually tracked in the junior bucket. After that mismatch exists, a later senior loss can attempt to settle more SY out of the junior bucket than the junior bucket contains, causing `update_tranching_market` to fail.

In `update_tranching_market`, senior losses are handled first. The coverage amount is later settled by moving SY from the junior bucket to the senior bucket.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
096 | pub fn update_tranching_market<M: JuniorReturnShareModel>(
097 |     market: &mut ExponentTranchingMarket,
098 |     return_model: &mut M,
099 |     senior_sy_amount: u64,
100 |     junior_sy_amount: u64,
101 |     sy_exchange_rate: Number,
102 |     current_timestamp: i64,
103 | ) -> Result<MarketUpdateAccruals> {
126 |     let senior_loss_coverage_applied =
127 |         apply_senior_side_loss_coverage_with_junior_fee_recalculation(
128 |             &mut market.financials,
129 |             senior_loss,
130 |             &mut junior_gain_accrual,
131 |             market.protocol_fee_config.jr_protocol_fee,
132 |             market.risk_config.jr_net_asset_dust_tolerance,
133 |         )?;
134 |     settle_senior_loss_coverage(
135 |         &mut settled_senior_sy_amount,
136 |         &mut settled_junior_sy_amount,
137 |         senior_loss_coverage_applied,
138 |         sy_exchange_rate,
139 |     )?;
141 |     let distributable_senior_gain =
142 |         apply_senior_side_gain_recovery(&mut market.financials, senior_gain)?;
```

The problem is in the recovery path. When senior-side gains recover junior impermanent loss, only `jr_effective_net_asset` and `jr_impermanent_loss` are updated. No SY is moved from the senior bucket back to the junior bucket.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
522 | pub fn apply_senior_side_gain_recovery(
523 |     financials: &mut TranchingMarketFinancials,
```

```

524 |     senior_gain: Number,
525 | ) -> Result<Number> {
526 |     let senior_gain_after_senior_recovery = checked_sub_number(senior_gain, recover_senior)?;
527 |     let recover_junior = senior_gain_after_senior_recovery.min(financials.jr_impermanent_loss);
528 |     financials.jr_effective_net_asset =
529 |         checked_add_number(financials.jr_effective_net_asset, recover_junior)?;
530 |     financials.jr_impermanent_loss =
531 |         checked_sub_number(financials.jr_impermanent_loss, recover_junior)?;
532 |
533 |     checked_sub_number(senior_gain_after_senior_recovery, recover_junior)
534 | }

```

This can leave:

```
jr_effective_net_asset > jr_raw_net_asset
```

That means part of the junior tranche's value is effectively backed by SY sitting in the senior bucket.

On a later SY price decline, senior loss coverage is capped by `jr_effective_net_asset`, not by the junior bucket's raw NAV or `junior_sy_amount`.

```

/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
342 | fn apply_senior_side_loss_coverage_with_junior_fee_recalculation(
343 |     financials: &mut TranchingMarketFinancials,
344 |     senior_loss: Number,
345 |     junior_gain_accrual: &mut JuniorGainAccrual,
346 |     junior_protocol_fee: Number,
347 |     junior_nav_dust_tolerance: Number,
348 | ) -> Result<Number> {
349 |     let coverage_applied = senior_loss.min(financials.jr_effective_net_asset);
350 |     if coverage_applied != Number::ZERO {
351 |         financials.jr_effective_net_asset =
352 |             checked_sub_number(financials.jr_effective_net_asset, coverage_applied)?;
353 |         financials.jr_impermanent_loss =
354 |             checked_add_number(financials.jr_impermanent_loss, coverage_applied)?;
355 |     }
356 |     Ok(coverage_applied)
357 | }

```

But settlement still tries to source the full coverage amount from `junior_sy_amount`.

```

/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
380 | fn settle_senior_loss_coverage(
381 |     senior_sy_amount: &mut u64,
382 |     junior_sy_amount: &mut u64,
383 |     coverage_applied: Number,
384 |     sy_exchange_rate: Number,
385 | ) -> Result<()> {
386 |     transfer_settled_sy(
387 |         junior_sy_amount,
388 |         senior_sy_amount,
389 |         coverage_applied,
390 |         sy_exchange_rate,

```

```
391 |     )
392 | }
```

If `coverage_applied / sy_exchange_rate` is greater than the junior bucket's SY balance, this subtraction fails.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
408 | fn transfer_settled_sy(
409 |     source_sy_amount: &mut u64,
410 |     destination_sy_amount: &mut u64,
411 |     nav_amount: Number,
412 |     sy_exchange_rate: Number,
413 | ) -> Result<()> {
414 |     let delta_sy = convert_nav_to_sy_amount(nav_amount, sy_exchange_rate)?;
415 |     if delta_sy == 0 {
416 |         return Ok(());
417 |     }
418 |     *source_sy_amount = source_sy_amount
419 |         .checked_sub(delta_sy)
420 |         .ok_or(ExponentTranchingError::MathOverflow)?;
421 | }
```

Since deposits and withdrawals both call `update_tranching_market` before proceeding, so this failure blocks those flows.

Example

- Initial state:

```
senior_sy_amount = 1000
junior_sy_amount = 500
sy_exchange_rate = 1.0

sr_raw_net_asset = 1000
jr_raw_net_asset = 500
sr_effective_net_asset = 1000
jr_effective_net_asset = 500
```

- Step 1: SY price drops to `0.9`.

```
senior raw NAV: 1000 -> 900, senior_loss = 100
junior raw NAV: 500 -> 450, junior_loss = 50

junior absorbs its own 50 loss:
jr_effective_net_asset = 450

junior then covers senior's 100 loss:
jr_effective_net_asset = 350
jr_impermanent_loss = 100

settlement moves floor(100 / 0.9) = 111 SY from junior to senior:
```

```
senior_sy_amount = 1111
junior_sy_amount = 389
```

- Step 2: SY price recovers to 1.0.

```
senior raw NAV increases, producing `senior_gain`

apply_senior_side_gain_recovery uses 100 of that gain to recover junior IL:
jr_effective_net_asset increases by 100
jr_impermanent_loss becomes 0

but no SY is moved back from senior to junior
```

Result:

```
junior_sy_amount = 389
jr_raw_net_asset = 389
jr_effective_net_asset is about 489

jr_effective_net_asset > jr_raw_net_asset
```

- Step 3: SY price drops to 0.5.

```
senior_loss is large
coverage_applied is capped by jr_effective_net_asset
coverage_applied can exceed the NAV backed by junior_sy_amount

settlement tries to transfer floor(coverage_applied / 0.5) SY from junior to senior
but junior only has 389 SY
checked_sub fails with MathOverflow
```

PoC

```
#[test]
fn senior_loss_coverage_can_exceed_junior_sy_after_recovery() {
    let mut financials = zeroed_financials();
    financials.sr_raw_net_asset = Number::from_natural_u64(1_000);
    financials.jr_raw_net_asset = Number::from_natural_u64(500);
    financials.sr_effective_net_asset = Number::from_natural_u64(1_000);
    financials.jr_effective_net_asset = Number::from_natural_u64(500);
    let risk_config = TranchingRiskConfig {
        liquidation_utilization: Number::from_natural_u64(10),
        ..default_risk_config()
    };
    let mut market = test_market(TranchingMarketState::Active, financials, risk_config);
    market.tranche_asset_state.senior_sy_amount = 1_000;
    market.tranche_asset_state.junior_sy_amount = 500;
    let mut return_model = FixedJuniorReturnShareModel(Number::ZERO);
    update_tranching_market(
        &mut market,
        &mut return_model,
        1_000,
        500,
    );
}
```



```

        Number::from_ratio(9, 10),
        1,
    ).unwrap();
    let senior_sy_amount = market.tranche_asset_state.senior_sy_amount;
    let junior_sy_amount = market.tranche_asset_state.junior_sy_amount;
    update_tranching_market(
        &mut market,
        &mut return_model,
        senior_sy_amount,
        junior_sy_amount,
        UNIT,
        2,
    ).unwrap();
    assert!(market.financials.jr_effective_net_asset > market.financials.jr_raw_net_asset);
    let senior_sy_amount = market.tranche_asset_state.senior_sy_amount;
    let junior_sy_amount = market.tranche_asset_state.junior_sy_amount;
    let result = update_tranching_market(
        &mut market,
        &mut return_model,
        senior_sy_amount,
        junior_sy_amount,
        Number::from_ratio(1, 2),
        3,
    );
    assert!(result.is_err());
}

```

Resolution

Resolved by commit [077f3dc](#).

EXPONENT_TRANCHING

[L-02] First accrued junior return share can be incorrectly reset

Identified in commit [b45838d](#).

In `update_tranching_market`, the protocol derives the `junior_return_share_ratio` from the time weighted accumulated value `tw_junior_return_share_accrued`.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
143 | let junior_return_share_ratio = calculate_distribution_junior_return_share(
144 |     &market.financials,
145 |     instant_junior_return_share_ratio,
146 |     current_timestamp,
147 | )?;

281 | fn calculate_distribution_junior_return_share(
282 |     financials: &TranchingMarketFinancials,
283 |     instant_junior_return_share_ratio: Number,
284 |     current_timestamp: i64,
285 | ) -> Result<Number> {
286 |     let elapsed_seconds =
287 |         calculate_elapsed_seconds(financials.last_distribution_ts, current_timestamp);
288 |     if elapsed_seconds == 0 || financials.tw_junior_return_share_accrued == Number::ZERO {
289 |         return Ok(instant_junior_return_share_ratio);
290 |     }
291 |
292 |     checked_div_number_u128(financials.tw_junior_return_share_accrued, elapsed_seconds)
293 | }
```

In `calculate_distribution_junior_return_share`, the elapsed time since `financials.last_distribution_ts` is computed via `calculate_elapsed_seconds`. If `last_distribution_ts == 0`, the function returns `0` directly, causing `calculate_distribution_junior_return_share` to fall back to the instantaneous `instant_junior_return_share_ratio` as the final `junior_return_share_ratio`.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
1209 | fn calculate_elapsed_seconds(last_timestamp: i64, current_timestamp: i64) -> u128 {
1210 |     if last_timestamp <= 0 || current_timestamp <= last_timestamp {
1211 |         return 0;
1212 |     }
1213 |
1214 |     (current_timestamp - last_timestamp) as u128
1215 | }
```

However, when a user deposits `SY` tokens into the market for the first time in `update_tranching_market`, `financials.last_distribution_ts` is not updated to the current timestamp or aligned with `last_sync_ts`. Because no distribution has occurred yet, `financials.last_distribution_ts` remains `0`. Later, once `accrued_junior_return_share` has accumulated, in the `update_tranching_market` process the ratio calculation still treats `last_distribution_ts` as `0`, directly uses the instantaneous `junior_return_share`

`_ratio`, skips the accumulated `accrued_junior_return_share`, and then resets it.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
189 | market.financials.last_sync_ts = current_timestamp;
190 |
191 | if distribution.fee_eligible {
192 |     market.financials.last_distribution_ts = current_timestamp;
193 |     market.financials.tw_junior_return_share_accrued = Number::ZERO;
194 | }
```

As a result, the first period of `accrued_junior_return_share` in the market is incorrectly discarded, and an incorrect `junior_return_share_ratio` is used.

It is recommended to align `last_distribution_ts` with `last_sync_ts` when the first `SY` deposit is made into the market.

Resolution

Resolved by commit `0d09ee1`.

EXPONENT_TRANCHING

[L-03] Accumulation of rounding dust can cause DoS

Identified in commit [b45838d](#).

Within the `update_tranching_market` instruction, the protocol distributes residual senior gains to the junior tranche by first crediting the junior's effective NAV, and subsequently settling the physical equivalent in SY tokens from the senior bucket to the junior bucket. However, an accounting discrepancy between the nominal NAV calculation and the integer-based token settlement allows rounding dust to accumulate, artificially inflating the junior tranche's effective NAV and creating a severe Denial of Service (DoS) vector during market downturns.

During an upward sync, `distribute_residual_senior_gain` credits the junior tranche with the exact, unrounded fractional value of their `junior_return_share`:

```
/* src/state/exponent_tranching_market/helper.rs */
564 |     financials.current_junior_return_share = junior_return_share_ratio;
565 |     financials.jr_effective_net_asset =
566 |         checked_add_number(financials.jr_effective_net_asset, junior_return_share)?;
```

However, the subsequent `settle_junior_return_share` function converts this nominal NAV into a physical SY token amount, strictly flooring the result to avoid fractional tokens:

```
/* src/state/exponent_tranching_market/helper.rs */
1217 | fn convert_nav_to_sy_amount(nav: Number, sy_exchange_rate: Number) -> Result<u64> {
1223 |     let amount = nav
1224 |         .checked_div(&sy_exchange_rate)
1225 |         .ok_or(ExponentTranchingError::MathOverflow)?
1226 |         .floor_u128();
```

Because the nominal NAV is fully credited but the physical SY transfer is floored, the junior tranche receives effective NAV that is not strictly backed by settled SY. Over repeated rising-rate synchronizations, this unbacked fractional dust accumulates, leading to a state divergence where `jr_effective_net_asset > jr_raw_net_asset`.

This divergence becomes fatal when the SY exchange rate experiences a sharp decline. During a senior loss event, the protocol utilizes the junior's effective NAV as the maximum cap for loss coverage:

```
/* src/state/exponent_tranching_market/helper.rs */
353 |     let coverage_applied = senior_loss.min(financials.jr_effective_net_asset);
```

When the protocol attempts to settle this loss coverage by transferring SY back to the senior bucket (`set`

`tle_senior_loss_coverage`), it assumes the junior bucket physically holds enough SY to cover its entire `effective_net_asset`. Because the junior NAV contains accumulated, unsettled dust, the required SY transfer can exceed the actual `junior_sy_amount`. This causes a fatal underflow during the token subtraction step:

```
/* src/state/exponent_tranching_market/helper.rs */
423 |     *source_sy_amount = source_sy_amount
424 |     .checked_sub(delta_sy)
425 |     .ok_or(ExponentTranchingError::MathOverflow)?;
```

Because both `deposit` and `withdraw` instructions invoke `update_tranching_market` as their very first action, hitting this `MathOverflow` panic will instantly brick the market, rendering users unable to deposit or withdraw funds until the underlying exchange rate shifts out of the failing condition.

PoC

```
#[test]
fn settlement_floor_dust_does_not_brick_loss_coverage() {
    let mut financials = zeroed_financials();
    financials.sr_raw_net_asset = Number::from_natural_u64(1_000_000);
    financials.jr_raw_net_asset = Number::from_natural_u64(100_000);
    financials.sr_effective_net_asset = Number::from_natural_u64(1_000_000);
    financials.jr_effective_net_asset = Number::from_natural_u64(100_000);
    financials.last_sync_ts = 1;
    financials.last_distribution_ts = 1;

    let risk_config = TranchingRiskConfig {
        beta: UNIT,
        liquidation_utilization: Number::from_natural_u64(1_000),
        fixed_term_duration_sec: 0,
        ..default_risk_config()
    };

    let mut market = test_market(TranchingMarketState::Active, financials, risk_config);
    market.tranche_asset_state.senior_sy_amount = 1_000_000;
    market.tranche_asset_state.junior_sy_amount = 100_000;

    let mut return_model = FixedJuniorReturnShareModel(Number::from_ratio(2, 5));

    let mut ts: i64 = 2;
    for i in 0..8u128 {
        let s = market.tranche_asset_state.senior_sy_amount;
        let j = market.tranche_asset_state.junior_sy_amount;
        let rate = Number::from_ratio(101 + i, 100);
        update_tranching_market(&mut market, &mut return_model, s, j, rate, ts)
            .expect("rising-rate sync should succeed");
        ts += 1;
    }

    let divergence = market
        .financials
        .jr_effective_net_asset
        .checked_sub(&market.financials.jr_raw_net_asset)
        .expect("jr_eff >= jr_raw");
    println!("accumulated jr_eff - jr_raw = {:?}", divergence);
}
```

```
let s = market.tranche_asset_state.senior_sy_amount;
let j = market.tranche_asset_state.junior_sy_amount;
let crash_rate = Number::from_ratio(1, 2);
let result = update_tranching_market(&mut market, &mut return_model, s, j, crash_rate, ts);

assert!(
    result.is_ok(),
    "accumulated floor dust must not underflow loss-coverage settlement"
);
}
```

Resolution

Resolved by commit [077f3dc](#).

EXPONENT_TRANCHING

[I-01] Small dust deposits and withdrawals can distort LP price

Identified in commit [b45838d](#).

The `deposit` and `withdraw` flows both rely on floor rounding when converting NAV into LP shares or asset claims. In the dust region, this allows a small operation to succeed while the state transition becomes asymmetric: a deposit may add tranche NAV while minting `0` LP shares, and a direct withdrawal may burn LP shares while returning `0` assets.

```
/* solana/programs/exponent_tranching/src/instructions/deposit.rs */
395 | fn calculate_lp_out(
396 |     value_allocated: Number,
397 |     current_lp_supply: u64,
398 |     nav_to_mint_lp_at: Number,
399 | ) -> Result<u64> {
400 |     let virtual_total_supply = (current_lp_supply as u128)
401 |         .checked_add(1)
402 |         .ok_or(ExponentTranchingError::MathOverflow)?;
403 |     let virtual_total_assets = checked_add_number(nav_to_mint_lp_at, UNIT)?;
404 |     let shares_out = value_allocated
405 |         .checked_mul(&Number::from(virtual_total_supply))
406 |         .and_then(|value| value.checked_div(&virtual_total_assets))
407 |         .ok_or(ExponentTranchingError::MathOverflow)?
408 |         .floor_u128();
409 |
410 |     shares_out
411 |         .try_into()
412 |         .map_err(|_| ExponentTranchingError::MathOverflow.into())
413 | }
```

The `calculate_deposit_quote` function only restricts `lp_out >= params.min_lp_out`. If `min_lp_out == 0`, then `lp_out` is allowed to be 0.

```
/* solana/programs/exponent_tranching/src/instructions/deposit.rs */
351 | fn calculate_deposit_quote(
352 |     market: &ExponentTranchingMarket,
353 |     params: &DepositParams,
354 |     current_lp_supply: u64,
355 |     sy_exchange_rate: Number,
356 | ) -> Result<DepositQuote> {
364 |     let gross_lp_out = calculate_lp_out(value_allocated, current_lp_supply, nav_to_mint_lp_at)?;
365 |     let protocol_fee_lp_shares =
366 |         calculate_deposit_fee_lp_shares(market, params.tranche_side, gross_lp_out)?;
367 |     let lp_out = gross_lp_out
368 |         .checked_sub(protocol_fee_lp_shares)
369 |         .ok_or(ExponentTranchingError::MathOverflow)?;
370 |     require!(
371 |         lp_out >= params.min_lp_out,
372 |         ExponentTranchingError::MinLpOutNotMet
373 |     );
```

Because `calculate_tranche_lp_prices_net_asset` derives the recorded LP price from effective NAV and tracked LP supply, these dust operations can move the reported LP price without a proportional user claim update.

```
/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
847 | pub fn calculate_tranche_lp_prices_net_asset(
848 |     market: &ExponentTranchingMarket,
849 | ) -> Result<TrancheLpPrices> {
850 |     Ok(TrancheLpPrices {
851 |         senior_lp_price_net_asset: calculate_lp_price_net_asset(
852 |             market.tranche_supply_state.total_senior_lp_supply,
853 |             market.financials.sr_effective_net_asset,
854 |         )?,
855 |         junior_lp_price_net_asset: calculate_lp_price_net_asset(
856 |             market.tranche_supply_state.total_junior_lp_supply,
857 |             market.financials.jr_effective_net_asset,
858 |         )?,
859 |     })
860 | }
```

This issue can slightly but repeatedly distort the market-observed LP price. A dust deposit can raise LP price without minting any LP to the depositor, and a dust direct withdrawal can reduce tracked LP supply without returning assets. Over time, repeated dust interactions can bias pricing, create donation-like value transfers to incumbent LP holders, and make downstream accounting or integrations observe a price move that is not backed by a proportional share change.

Consider rejecting dust operations whenever the user-side state change rounds to zero. In practice, the deposit path should require positive `lp_out`, and the direct withdraw path should require positive `amount_out`, even when the caller passes a zero minimum. The withdrawal side is partially mitigated because `wrapper_withdraw` already rejects zero-output redemptions. It may be worth adding a similar guard to the deposit path so zero-LP dust deposits are also rejected.

A test case example:

```
#[test]
fn repeated_zero_lp_deposits_monotonically_increase_lp_price() {
    let mut market = test_market(TranchingMarketState::Active);
    market.financials.jr_raw_net_asset = Number::from_natural_u64(2_000);
    market.financials.jr_effective_net_asset = Number::from_natural_u64(2_000);
    market.tranche_asset_state.junior_sy_amount = 2_000;
    market.tranche_supply_state.total_junior_lp_supply = 1_000;

    let params = DepositParams {
        tranche_side: TrancheSide::Junior,
        amount_in: 1,
        min_lp_out: 0,
    };
    let sy_exchange_rate = Number::from_natural_u64(1);
```



```

let mut previous_price = calculate_tranche_lp_prices_net_asset(&market)
    .unwrap()
    .junior_lp_price_net_asset;

for expected_effective_nav in 2_001_u64..=2_100 {
    let current_lp_supply =
        selected_total_lp_supply(&market.tranche_supply_state, params.tranche_side);
    let quote =
        calculate_deposit_quote(&market, &params, current_lp_supply, sy_exchange_rate).unwrap();

    assert_eq!(quote.lp_out, 0);
    assert_eq!(quote.new_total_lp_supply, current_lp_supply);

    apply_deposit_to_market(&mut market, &params, &quote).unwrap();

    assert_eq!(market.tranche_supply_state.total_junior_lp_supply, 1_000);
    assert_eq!(market.tranche_asset_state.junior_sy_amount, expected_effective_nav);
    assert_eq!(
        market.financials.jr_effective_net_asset,
        Number::from_natural_u64(expected_effective_nav)
    );

    let next_price = calculate_tranche_lp_prices_net_asset(&market)
        .unwrap()
        .junior_lp_price_net_asset;
    assert!(next_price > previous_price);

    previous_price = next_price;
}
}

```

Resolution

This issue has been fixed by [711a77d](#).

EXPONENT_TRANCHING

[I-02] Circumventing fees by splitting withdrawals

Identified in commit [b45838d](#).

The `deposit` and `withdraw` flows both compute transfer fees in LP shares and then round those fees down with `floor_u128`. When a user splits a larger operation into sufficiently small pieces, each individual fee can round down to zero while the user still receives a positive LP mint or asset redemption. As a result, the protocol can collect less total fee than intended for the same aggregate deposit or withdrawal size.

```
/* solana/programs/exponent_tranching/src/instructions/deposit.rs */
351 | fn calculate_deposit_quote(
352 |     market: &ExponentTranchingMarket,
353 |     params: &DepositParams,
354 |     current_lp_supply: u64,
355 |     sy_exchange_rate: Number,
356 | ) -> Result<DepositQuote> {
365 |     let protocol_fee_lp_shares =
366 |         calculate_deposit_fee_lp_shares(market, params.tranche_side, gross_lp_out)?;
367 |     let lp_out = gross_lp_out
368 |         .checked_sub(protocol_fee_lp_shares)
369 |         .ok_or(ExponentTranchingError::MathOverflow)?;

/* solana/programs/exponent_tranching/src/instructions/withdraw.rs */
347 | fn calculate_withdraw_quote(
348 |     market: &ExponentTranchingMarket,
349 |     params: &WithdrawParams,
350 |     current_lp_supply: u64,
351 |     sy_exchange_rate: Number,
352 | ) -> Result<WithdrawQuote> {
360 |     let protocol_fee_lp_shares =
361 |         calculate_withdraw_fee_lp_shares(market, params.tranche_side, params.lp_amount_in)?;
362 |     let redeem_lp_shares = params
363 |         .lp_amount_in
364 |         .checked_sub(protocol_fee_lp_shares)
365 |         .ok_or(ExponentTranchingError::MathOverflow)?;

/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
697 | pub fn calculate_deposit_fee_lp_shares(
698 |     market: &ExponentTranchingMarket,
699 |     tranche_side: TrancheSide,
700 |     lp_shares: u64,
701 | ) -> Result<u64> {
702 |     calculate_transfer_fee_lp_shares(
703 |         market,
704 |         tranche_side,
705 |         TrancheTransferKind::Deposit,
706 |         lp_shares,
707 |     )
708 | }

/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
710 | pub fn calculate_withdraw_fee_lp_shares(
711 |     market: &ExponentTranchingMarket,
```

```

712 |     tranche_side: TrancheSide,
713 |     lp_shares: u64,
714 | ) -> Result<u64> {
715 |     calculate_transfer_fee_lp_shares(
716 |         market,
717 |         tranche_side,
718 |         TrancheTransferKind::Withdraw,
719 |         lp_shares,
720 |     )
721 | }

/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
723 | fn calculate_transfer_fee_lp_shares(
724 |     market: &ExponentTranchingMarket,
725 |     tranche_side: TrancheSide,
726 |     transfer_kind: TrancheTransferKind,
727 |     lp_shares: u64,
728 | ) -> Result<u64> {
729 |     calculate_tranche_fee_lp_shares(
730 |         lp_shares,
731 |         selected_transfer_fee(&market.protocol_fee_config, tranche_side, transfer_kind),
732 |     )
733 | }

/* solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs */
748 | pub fn calculate_tranche_fee_lp_shares(lp_shares: u64, fee: Number) -> Result<u64> {
749 |     let fee_lp_shares = fee
750 |         .checked_mul(&Number::from_natural_u64(lp_shares))
751 |         .ok_or(ExponentTranchingError::MathOverflow)?
752 |         .floor_u128(); // audit: floor that poses security risks
753 |
754 |     fee_lp_shares
755 |         .try_into()
756 |         .map_err(|_| ExponentTranchingError::MathOverflow.into())
757 | }

```

This issue allows fee erosion through both deposit splitting and withdrawal splitting. Here is a test case where a user splits a large withdraw operation into multiple small withdraw operations to bypass fees:

```

#[cfg(test)]
mod tests {
    use super::*;
    use crate::{
        cpi_common::CpiAccounts,
        state::{
            TrancheAssetState, TrancheSupplyState, TranchingMarketFinancials, TranchingMarketRoles,
            TranchingProtocolFeeConfig, TranchingRiskConfig, RETURN_MODEL_RESERVED_PADDING_SIZE,
        },
    };

    fn test_market(market_state: TranchingMarketState) -> ExponentTranchingMarket {
        ExponentTranchingMarket {
            address_lookup_table: Pubkey::default(),
            sy_mint: Pubkey::default(),
            sy_program: Pubkey::default(),
            token_sy_escrow: Pubkey::default(),
            mint_lp_senior: Pubkey::default(),
            mint_lp_junior: Pubkey::default(),

```

```

    self_address: Pubkey::default(),
    return_model_storage: Pubkey::default(),
    signer_bump: [0],
    return_model_storage_bump: [0],
    seed_id: [0; 8],
    status_flags: 0,
    market_state,
    financials: TranchingMarketFinancials::default(),
    tranche_supply_state: TrancheSupplyState::default(),
    tranche_asset_state: TrancheAssetState::default(),
    risk_config: TranchingRiskConfig::default(),
    protocol_fee_config: TranchingProtocolFeeConfig::default(),
    reserved_padding: [0; RETURN_MODEL_RESERVED_PADDING_SIZE],
    roles: TranchingMarketRoles::default(),
    sy_cpi_accounts: CpiAccounts::default(),
  }
}

#[test]
fn split_withdraws_can_avoid_floored_withdraw_fees() {
  let mut single_withdraw_market = test_market(TranchingMarketState::Active);
  single_withdraw_market.financials.jr_raw_net_asset = Number::from_natural_u64(100_000);
  single_withdraw_market.financials.jr_effective_net_asset = Number::from_natural_u64(100_000);
  single_withdraw_market.tranche_asset_state.junior_sy_amount = 100_000;
  single_withdraw_market.tranche_supply_state.total_junior_lp_supply = 1_000;
  single_withdraw_market.protocol_fee_config.junior_withdraw_protocol_fee =
    Number::from_ratio(1, 10);

  let mut split_withdraw_market = single_withdraw_market.clone();
  let sy_exchange_rate = Number::from_natural_u64(1);

  let single_params = WithdrawParams {
    tranche_side: TrancheSide::Junior,
    lp_amount_in: 10,
    min_amount_out: 0,
  };
  let single_quote = calculate_withdraw_quote(
    &single_withdraw_market,
    &single_params,
    single_withdraw_market.tranche_supply_state.total_junior_lp_supply,
    sy_exchange_rate,
  )
  .unwrap();

  assert_eq!(single_quote.protocol_fee_lp_shares, 1);
  assert_eq!(single_quote.amount_out, 899);

  let mut split_total_amount_out = 0_u64;
  let mut split_total_fee = 0_u64;

  for _ in 0..10 {
    let split_params = WithdrawParams {
      tranche_side: TrancheSide::Junior,
      lp_amount_in: 1,
      min_amount_out: 0,
    };
    let current_lp_supply = split_withdraw_market.tranche_supply_state.total_junior_lp_supply;
    let split_quote =
      calculate_withdraw_quote(&split_withdraw_market, &split_params, current_lp_supply,
        ↪ sy_exchange_rate)

```

```

        .unwrap();

        assert_eq!(split_quote.protocol_fee_lp_shares, 0);
        assert!(split_quote.amount_out > 0);

        split_total_amount_out += split_quote.amount_out;
        split_total_fee += split_quote.protocol_fee_lp_shares;
        apply_withdraw_to_market(&mut split_withdraw_market, &split_params, &split_quote).unwrap();
    }

    assert_eq!(split_total_fee, 0);
    assert!(split_total_amount_out >= single_quote.amount_out);
}
}

```

However, because the saved amount is measured in the smallest LP unit rather than a full LP, the practical economic impact appears relatively low.

Consider rounding up the fees for any non-zero LP-denominated transfer fee or enforcing a minimum non-zero fee whenever the operation produces a positive LP mint or positive asset redemption.

```

let fee_lp_shares = fee
    .checked_mul(&Number::from_natural_u64(lp_shares))
    .ok_or(ExponentTranchingError::MathOverflow)?
    .ceil_u128();

require!(
    fee == Number::ZERO || fee_lp_shares > 0,
    ExponentTranchingError::AmountTooSmall
);

```

Resolution

This issue has been fixed by [7c2effd](#).

EXPONENT_TRANCHING

[I-03] Inconsistencies in the ModifyMarket instruction

Identified in commit [b45838d](#).

There are two correctness and consistency items in the `ModifyMarket` instruction. Since `ModifyMarket` is a high-trust instruction, there is no immediate security impact.

1. ModifyMarket does not preserve ALT validation invariants

During initialization, `address_lookup_table` is explicitly constrained to be owned by the Solana address lookup table program. However, later reconfiguration through `modify_market` can replace `market.address_lookup_table` without re-applying the same invariant. This creates a cross-instruction inconsistency: the market is initialized under one ALT validation rule, but can later be updated to a value that would have been rejected at creation time.

```
167 | pub(crate) fn apply_modify_market_params(
168 |     market: &mut ExponentTranchingMarket,
169 |     return_model: &mut TrancheReturnModel,
170 |     params: ModifyMarketParams,
171 | ) -> Result<()> {
197 |     if let Some(address_lookup_table) = params.address_lookup_table {
198 |         market.address_lookup_table = address_lookup_table;
199 |     }

/* solana/programs/exponent_tranching/src/instructions/admin/initialize_tranching_market.rs */
042 | pub struct InitializeTranchingMarket<'info> {
067 |     #[account(owner = ADDRESS_LOOKUP_TABLE_PROGRAM_ID @ ExponentTranchingError::InvalidAddressLookupTable)]
068 |     pub address_lookup_table: UncheckedAccount<'info>,
```

An admin may unintentionally replace the market ALT with an account that does not satisfy the invariant enforced at initialization. The next market interaction that relies on ALT-based SY CPI resolution may fail.

Consider preserving the same ALT invariant in `modify_market` that is already enforced during initialization.

2. ModifyMarket can reconfigure SY source without first syncing market accounting

`modify_market` only performs `update_tranching_market(...)` when `risk_config`, `protocol_fee_config`, or `return_model` is being changed. Reconfiguring `sy_cpi_accounts` or `address_lookup_table` alone does not trigger the same synchronization step. As a result, the market can switch to a new SY state source while its stored `last_sync_ts`, raw NAV, effective NAV, utilization, and pending protocol fee state still reflect the old source.

```

/* solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs */
124 | fn requires_accounting_sync(params: &ModifyMarketParams) -> bool {
125 |     params.risk_config.is_some()
126 |     || params.protocol_fee_config.is_some()
127 |     || params.return_model.is_some()
128 | }

/* solana/programs/exponent_tranching/src/instructions/common.rs */
024 | pub(crate) fn load_sy_exchange_rate<'info>(
025 |     address_lookup_table: &AccountInfo<'info>,
026 |     market: &ExponentTranchingMarket,
027 |     account_pool: &[AccountInfo<'info>],
028 |     sy_program: &AccountInfo<'info>,
029 | ) -> Result<Number> {
030 |     let sy_state = do_get_sy_state(
031 |         address_lookup_table,
032 |         &market.sy_cpi_accounts,
033 |         account_pool,
034 |         sy_program,
035 |     )?;
036 |
037 |     Ok(sy_state.exchange_rate)
038 | }

```

This creates an accounting discontinuity across configuration updates. Before the reconfiguration, the market's stored accounting is still tied to the previous SY source. After the reconfiguration, the next `deposit`, `withdraw`, or syncing admin action will read the exchange rate from the new source and apply `update_tranching_market(...)` on top of stale accounting state that was never finalized against the old source first.

A routine operational change can leave the market with accounting that jumps directly from one SY source to another without an explicit boundary sync, making subsequent utilization, gain/loss recognition, and protocol fee accrual depend on an implicit source transition rather than a cleanly settled handoff.

Consider an accounting sync before applying any configuration change that can alter the SY state source used by `load_sy_exchange_rate(...)`, including `sy_cpi_accounts` and `address_lookup_table`.

Resolution

This issue has been fixed by [1222911](#).

EXPONENT_TRANCHING

[I-04] Suboptimal numerical integration for target drift calculation

Identified in commit [b45838d](#).

Within the utilization-guided return model, the protocol calculates the average target drift over a given time interval by approximating the area under the curve. The current implementation evaluates the `current_target`, the `new_target` at the end of the interval, and the `midpoint_target`, averaging them using the formula $(\text{current} + \text{new} + 2 * \text{midpoint}) / 4$.

```
/* src/state/exponent_tranching_market/return_model.rs */
878 | let new_target = calculate_shifted_target_point(current_target, linear_shift)?;
879 | let midpoint_target = calculate_shifted_target_point(current_target, linear_shift / 2)?;
880 | let midpoint_target_twice = number_raw_u128(midpoint_target)?
881 |     .checked_mul(2)
882 |     .ok_or(ExponentTranchingError::MathOverflow)?;
883 | let average_target_raw = number_raw_u128(current_target)?
884 |     .checked_add(number_raw_u128(new_target)?
885 |         .and_then(|value| value.checked_add(midpoint_target_twice)))
886 |     .ok_or(ExponentTranchingError::MathOverflow)?
887 |     / 4;
```

This mathematical approach is functionally equivalent to a two-segment composite trapezoidal rule. While computationally straightforward, the underlying target drift mechanism typically models compounding growth, mapping to an exponential curve in the form of $f(t) = \text{current} \cdot \exp(k \cdot t)$. Because exponential functions are strictly convex, applying linear trapezoidal segments systematically overestimates the true mathematical average across the interval.

To achieve a significantly higher degree of numerical precision with minimal additional computational overhead, it is recommended to upgrade the approximation to Simpson's rule. By shifting the mathematical weighting coefficients to $(\text{current} + \text{new} + 4 * \text{midpoint}) / 6$, the calculation evaluates a quadratic interpolation rather than a linear one. This provides a much tighter and more accurate approximation of the exponential integral, reducing systemic accumulation of mathematical drift over the protocol's lifecycle.

Resolution

Resolved by commit [d4f7539](#).

EXPONENT_TRANCHING

[I-05] Redundant Token-2022 extension check

Identified in commit [b45838d](#).

Within the `initialize_tranching_market` instruction, the protocol implements a specific security check, `validate_sy_mint_not_closeable`, to ensure that the provided Standard Yield (`sy_mint`) token cannot be maliciously closed. This function explicitly inspects the mint data for the `MintCloseAuthority` extension.

```
/* src/instructions/admin/initialize_tranching_market.rs */
087 | /// Use the old Token program as the implementation for SY and LP tokens
088 | pub token_program: Program<'info, Token>,
096 | fn validate_sy_mint_not_closeable(&self) -> Result<()> {
097 |     let sy_mint_info = self.sy_mint.to_account_info();
098 |     if *sy_mint_info.owner != token_2022::Token2022::id() {
099 |         return Ok(());
100 |     }
102 |     let mint_data = sy_mint_info.try_borrow_data()?;
103 |     let mint_state = StateWithExtensions::<spl_token_2022::state::Mint>::unpack(&mint_data)?;
104 |     if let Ok(close_authority) = mint_state.get_extension::<MintCloseAuthority>() {
105 |         let close_authority: Option<Pubkey> = close_authority.close_authority.into();
106 |         require!(
107 |             close_authority.is_none(),
108 |             ExponentTranchingError::CloseableSyMint
109 |         );
110 |     }
112 |     Ok(())
113 | }
```

However, the account validation struct for this instruction reveals that the protocol strictly enforces the use of the legacy SPL Token program, rather than supporting dynamic token resolution via the `TokenInterface`.

Because the `token_program` is hardcoded to `Program<'info, Token>`, any mint passed into the `sy_mint` account field must be a standard SPL Token. Consequently, the `sy_mint_info.owner` will never equal `token_2022::Token2022::id()`, rendering the entire `MintCloseAuthority` extension check unreachable and effectively redundant.

While removing this redundant code, a more relevant security consideration for standard SPL tokens is the `freeze_authority`. If the `sy_mint` retains an active `freeze_authority` controlled by a centralized or malicious entity, the protocol's tranches could be completely disabled. To align the security model with the enforced token standard, it is recommended to replace the Token-2022 extension check with a validation asserting that the `sy_mint`'s `freeze_authority` is either set to `None` or safely constrained to a trusted protocol actor.

Resolution

Resolved by commit [05d19b7](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Exponent Labs Ltd. (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

